Application Note

Application State Machine Application Note

Revision History

Date	Version	Description
September 29, 2021	0.50	Initial version created.

CONFIDENTIAL

©2021, Atmosic Technologies Inc. All rights reserved. Atmosic logo is a registered trademark of Atmosic Technologies Inc. All other trademarks are the properties of their respective holders. This document is subject to change without notice.

Table of Contents

1 Overview 3

2 Helper API 3

3 State Machine Table Structure 5

4 Multiple Operation Code in State 6

Appendix **7**

CONFIDENTIAL

List of Figures

Figure 1 - BLE_Adv State Machine Table
Figure 2 - Structure of State_entry
Figure 3 - State From APP_S_INIT to APP_S_IDLE
Figure 4 - Multiple Operation Code of S_ADV STARTED State
Figure 5 - BLE_ADV Example Complete State Machine Flow Chart

List of Tables

Table 1 - Application State Machine Helper API
Table 2 - Structure of State_entry

1 Overview

The Atmosic SDK framework provides the appropriate helper APIs to create application specific state machines. Figure 1 is the state machine table used in the BLE_Adv example.

```
static const state_entry s_tbl[] = {
    {S_OP(S_INIT, OP_MODULE_INIT), S_IDLE, ble_adv_init},
    {S_OP(S_IDLE, OP_START_ADV), S_ADV_STARTING, ble_adv_start_adv},
    {S_OP(S_ADV_STARTING, OP_START_ADV_CFM), S_ADV_STARTED, ble_adv_start_on},
    {S_OP(S_ADV_STARTED, OP_CONNECTED), S_CONNECTED, ble_adv_connected},
    {S_OP(S_CONNECTED, OP_DISCONNECTED), S_ADV_STOPPED, ble_adv_disconnected},
    {S_OP(S_CONNECTED, OP_ADV_TIMEOUT), S_CONNECTED, ble_adv_connected},
    {S_OP(S_ADV_STARTED, OP_ADV_TIMEOUT), S_ADV_STOPPED, ble_adv_timeout},
    {S_OP(S_ADV_STOPPED, OP_RESTART_ADV), S_ADV_STARTING, ble_adv_restart_adv}
};
```

Figure 1 - BLE_Adv State Machine Table

S_* : User define state code
OP_*: User define operation code
S_OP: It is a macros to use to define now state and operation code

Every item in s_tbl[] expresses how to manage, control, and restrict the state that can change to the other state by specific operation. Section 3 State Machine Table Structure provides more detail. Figure 5 in the Appendix section shows the complete flow chart of the BLE_ADV example state machine.

The application can define its own finite state machine and transition to the next state upon receiving an event.

2 Helper API

The following functions are exported by atm_asm.c It can be used to manage application state machines. See Table 1.

Function Name	Description
atm_asm_init_table	Insert application state machine table into framework
atm_asm_get_current_state	Get current state
atm_asm_set_current_state	Set state
atm_asm_set_state_op	Update current state and operation code
atm_asm_move	Move to handler function by operation code then update state
atm_asm_reg_state_change_cb	Set callback function to debug state machine transitions

Table 1 - Application State Machine Helper API

CONFIDENTIAL

The first input parameter of all of the functions is the index number of the state machine table instance. There is no limit on state machine table instances. The maximum instance number can be controlled by application code. atm_asm.cprovides below directives to define the instance number.

=

```
#ifndef CFG_S_MAX_ENTRY
#define CFG_S_MAX_ENTRY 1
#endif
```

User applications can overwrite this for their purpose. For example, a user can have one state machine table for Bluetooth LE and another state machine table for LED control. In this case, define CFG_S_MAX_ENTRY to 2.

CONFIDENTIAL

3 State Machine Table Structure

The following structure is defined in atm_asm.h, see Figure 2 and Table 2.

```
typedef struct {
    /// state code
    ASM_S state;
    /// operation code
    ASM_O operation;
    /// next state
    ASM_S next_state;
    /// handler function for state_operation
    state_handler handler;
} state_entry;
```

Figure 2 - Structure of State_entry

Member	Description
state	current state
operation	operation code
next_state	next state
handler	handler function for state_operation

Table 2 - Structure of State_entry

As shown in Figure 3, the following state transaction occurs when the current state is in S_INIT and receives OP_MODULE_INIT, the ble_adv_init() handler function will be called then move to the next S_IDLE state when calling atm_asm_move (APP_S_TBL_IDX, OP_MODULE_INIT).

```
{S_OP(S_INIT, OP_MODULE_INIT), S_IDLE, ble_adv_init},
```

CONFIDENTIAL

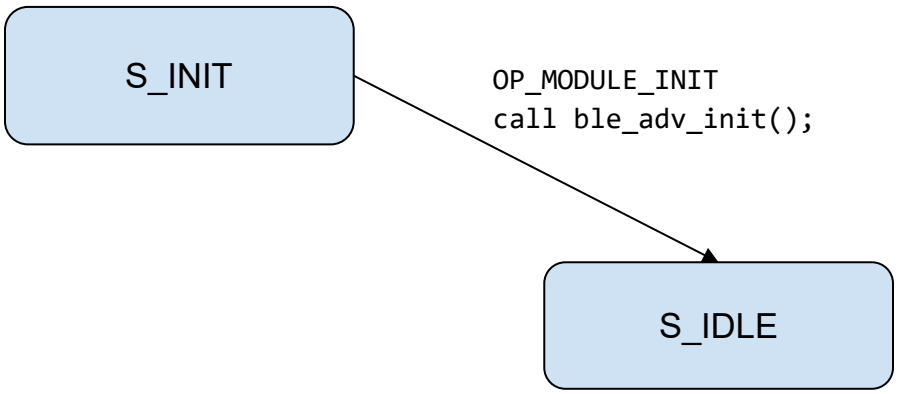


Figure 3 - State From APP_S_INIT to APP_S_IDLE

It means that it can only accept OP_MODULE_INIT operation code in S_INIT state. In most cases, application code will call the Atmosic SDK application API in a handler function (such as ble_adv_init) then get the result by callback function. In the callback function, use atm_asm_move API to move state then call handler function by operation code.

4 Multiple Operation Code in State

In some situations, application code may require transitioning to different states depending on a predefined condition. . In the example of BLE_adv, the application will conditionally change states depending on the operation code. . Taking BLE_adv as an example (see Figure 1 for the defined FSM), Figure 4 describes the state transaction flowchart when the current state is in S_ADV_STARTED and may receive OP_ADV_TIMEOUT or OP_CONNECTED operation code.

{S_OP(S_ADV_STARTED, OP_CONNECTED), S_CONNECTED, ble_adv_connected},
{S_OP(S_ADV_STARTED, OP_ADV_TIMEOUT), S_ADV_STOPPED, ble_adv_timeout},

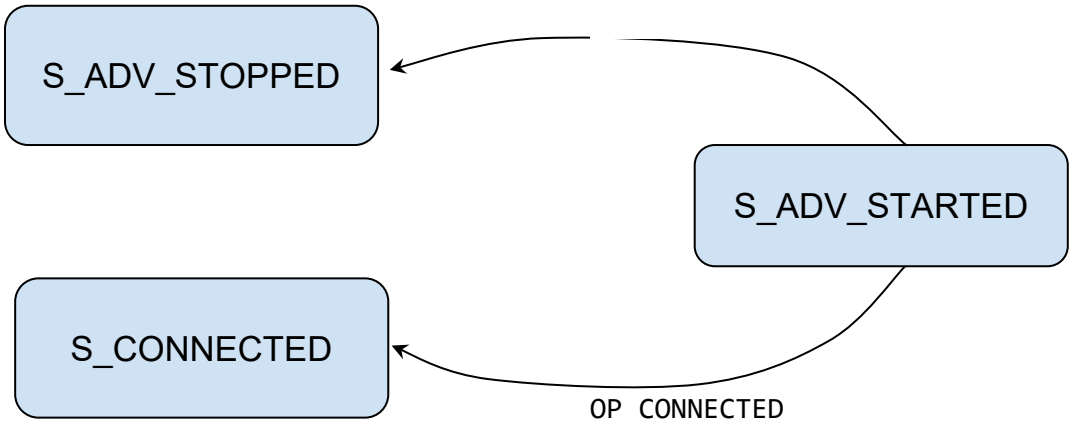


Figure 4 - Multiple Operation Code of S_ADV_STARTED State

CONFIDENTIAL

Appendix

BLE_adv Example Complete State Machine Flow Chart

Figure 1 shows the BLE_adv example state machine table, and Figure 5 is the complete flow chart of the BLE_adv example.

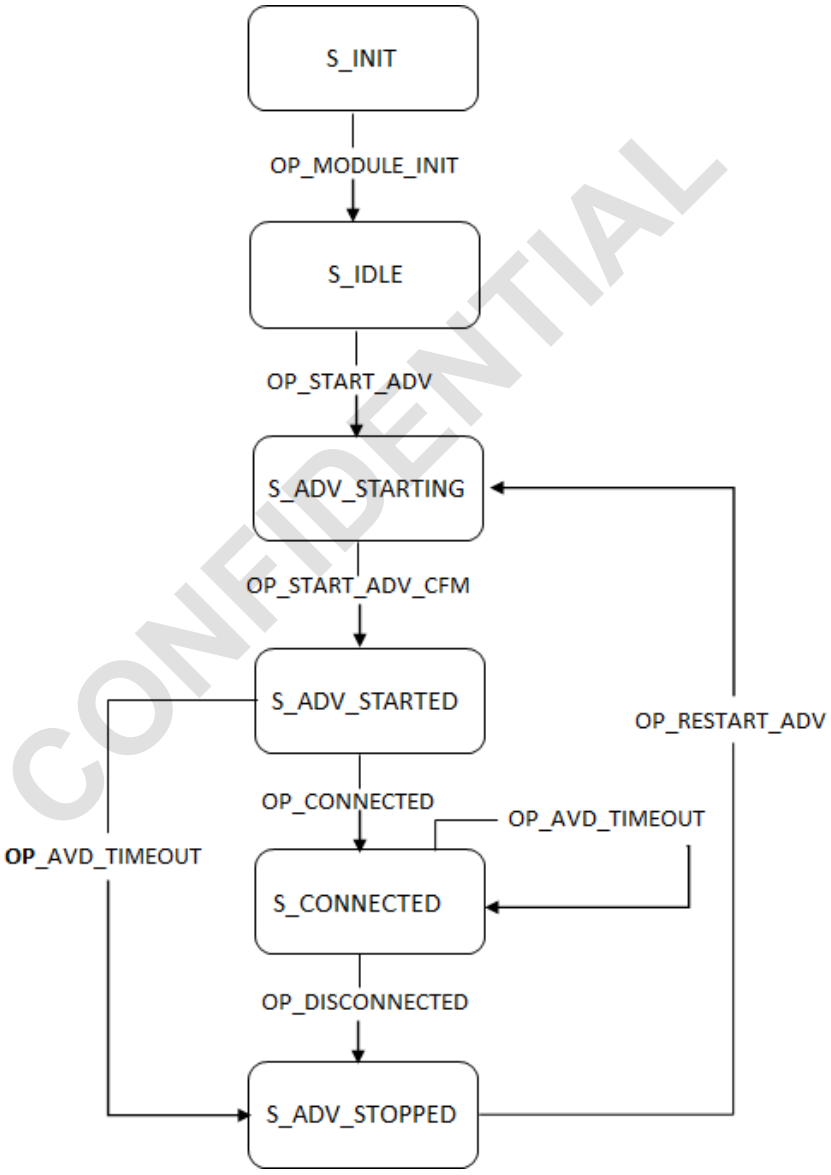


Figure 5 - BLE_ADV Example Complete State Machine Flow Chart

CONFIDENTIAL



CONFIDENTIAL

ATMOSIC TECHNOLOGIES - DISCLAIMER

This product document is intended to be a general informational aid and not a substitute for any literature or labeling accompanying your purchase of the Atmosic product. Atmosic reserves the right to amend its product literature at any time without notice and for any reason, including to improve product design or function. While Atmosic strives to make its documents accurate and current, Atmosic makes no warranty or representation that the information contained in this document is completely accurate, and Atmosic hereby disclaims (i) any and all liability for any errors or inaccuracies contained in any document or in any other product literature and any damages or lost profits resulting therefrom; (ii) any and all liability and responsibility for any action you take or fail to take based on the information contained in this document; and (iii) any and all implied warranties which may attach to this document, including warranties of fitness for particular purpose, non-infringement and merchantability. Consequently, you assume all risk in your use of this document, the Atmosic product, and in any action you take or fail to take based upon the information in this document. Any statements in this document in regard to the suitability of an Atmosic product for certain types of applications are based on Atmosic's general knowledge of typical requirements in generic applications and are not binding statements about the suitability of Atmosic products for any particular application. It is your responsibility as the customer to validate that a particular Atmosic product is suitable for use in a particular application. All content in this document is proprietary, copyrighted, and owned or licensed by Atmosic, and any unauthorized use of content or trademarks contained herein is strictly prohibited.

Copyright ©2021 by Atmosic Technologies. All rights reserved.

www.atmosic.com